

מדריך למידה ופיתוח: PlaylistSample

חזרה מעמיקה על MVVM, חיבור נתונים (Data Binding), פקודות ותשתית שירות מוק סטטי

יעד: מגמת הנדסת תוכנה / תיכון

ארכיטקטורה: MVVM נקי

פלטפורמה: .NET MAUI / NET 10

חלק 1: בניית האפליקציה שלב אחר שלב

1. המטרות הפדגוגיות של השלב

בשלב זה אנו מבצעים חזרה יסודית על ארכיטקטורת **MVVM** (Model-View-ViewModel). כדי להתרכז בהבנה עמוקה של העקרונות ללא רעשי רקע, ניקינו את הפרויקט מכל מורכבות מיותרת:

- **ללא Shell Navigation**: חלון האפליקציה טוען ישירות את הדף `MainPage`.
- **ללא DI Container (מכולת תלויות)**: החיבור בין הדף, ה-`ViewModel` וה-`Service` מתבצע ישירות ושקוף בקוד.
- **ללא Styles ו-Themes מורכבים**: ממשק XAML נקי וישיר שקל לקרוא ולתחזק.
- **תצוגת פריט בודד ודפדוף**: טרם הצגת רשימות ואוספים (`ObservableCollection`), אנו מתנסים בניהול פריט בודד על המסך עם כפתורי "הבא" ו"הקודם", מה שמכין את הקרקע לצורך המובהק באוספים בשלב הבא!

תרשים זרימת הנתונים והאחריות (Data Flow)



הפרדה מוחלטת: ה-`View` אינו יודע על ה-`Service`, וה-`Service` אינו מכיר את פקדי ה-`UI`.

2. שלב אחר שלב: בניית האפליקציה מאפס

שלב 1 הגדרת נקודת הכניסה לאפליקציה (ללא Shell)

במקום להשתמש במנגנון ה-Shell המורכב, נגדיר ב- `App.xaml.cs` טעינה ישירה של דף הבית:

```
namespace PlayListSample;

public partial class App : Application
{
    public App()
    {
        InitializeComponent();

        protected override Window CreateWindow(IActivationState? activationState)
        {
            // וללא ניתובים מיותרים Shell ללא MainPage החלון מייצר ישירות את
            return new Window(new MainPage());
        }
    }
}
```

דגש פדגוגי: 💡

הסרת ה-Shell מאפשרת לתלמידים להתרכז בליבה של חיבור הנתונים (Data Binding) מבלי להתבלבל מניתובים (Routes), טאבים או Flyouts.

המודל הוא מחלקת נתונים פשוטה (POCO). שימו לב שכללנו מזהה **Id** כדי להכין מראש את המודל למעבר עתידי ל-:SQLite

```
namespace PlayListSample.Models;

public class VideoItem
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Url { get; set; } = string.Empty;
    public string ChannelName { get; set; } = string.Empty;
    public int DurationMinutes { get; set; }
    public bool IsWatched { get; set; }

    // XAML-מאפייני עזר לקריאה בלבד עבור תצוגה נקייה ב
    public string FormattedDuration => DurationMinutes > 0 ? $"{DurationMinutes} צוין" : "0";
    public string WatchedStatusText => IsWatched ? "נצפה ✓" : "טרם נצפה";
}
```

שלב 3 שכבת השירות: החוזה והמוק הסטטי

אנו מגדירים ממשק `IVideoService` שכל מתודותיו א-סינכרוניות ומחזירות `Task`:

```
public interface IVideoService
{
    Task<IReadOnlyList<VideoItem>> GetAllVideosAsync();
    Task<VideoItem?> GetVideoByIdAsync(int id);
    Task<bool> AddVideoAsync(VideoItem video);
    Task<bool> UpdateVideoAsync(VideoItem video);
    Task<bool> DeleteVideoAsync(int id);
}
```

את הממשק אנו מממשים באמצעות `MockVideoService` עם רשימה סטטית פנימית ו-`await Task.Delay(100)` המדמה קריאה לרשת או לדיסק.

⚠ עקרון אדריכלי מכריע:

מדוע להשתמש ב-`Task` גם כשמדובר ברשימה בזיכרון? מכיוון שבעתיד נחליף את המוק במסד נתונים של SQLite, וה-`ViewModel` לא יצטרך להשתנות כלל!

כל ViewModel יורש ממחלקה זו, המממשת את `INotifyPropertyChanged` ומאפשרת למטכ לקבל עדכונים חיים על שינויי ערכים:

```
public abstract class BaseViewModel : INotifyPropertyChanged
{
    private bool _isLoading;
    public event PropertyChangedEventHandler? PropertyChanged;

    public bool IsLoading
    {
        get => _isLoading;
        set
        {
            if (_isLoading != value)
            {
                _isLoading = value;
                OnPropertyChanged();
                OnPropertyChanged(nameof(IsNotLoading));
            }
        }
    }

    public bool IsNotLoading => !IsLoading;

    protected void OnPropertyChanged([CallerMemberName] string? propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

שלב 5 ה-ViewModel הראשי: MainViewModel.cs

ה-ViewModel מנהל את כל המצב והפעולות במסך:

- אינדקס דפדוף (`_currentIndex`): מצביע על הסרטון הנוכחי ברשימה.
- פקודות עם `CanExecute`: `NextCommand` ו- `PreviousCommand` נעלות אוטומטית כאשר מגיעים לקצה הרשימה בעזרת `ChangeCanExecute()`.
- טופס קלט (Two-Way Binding): שדות `InputTitle`, `InputUrl` וכו' להזנת סרטון חדש.
- פקודת שמירה (`SaveCommand`): מוסיפה את הסרטון ל-Service ומעבירה מיידית את הדפדוף לסרטון החדש.

```
// הגדרת הפקודות והתנאים שלהן בבנאי
NextCommand = new Command(ExecuteNext, CanExecuteNext);
PreviousCommand = new Command(ExecutePrevious, CanExecutePrevious);

private bool CanExecuteNext()
{
    return IsNotLoading && _videos.Count > 0 && _currentIndex < _videos.Count - 1;
}

private bool CanExecutePrevious()
{
    return IsNotLoading && _videos.Count > 0 && _currentIndex > 0;
}
```

6 שלב המסך (View): MainPage.xaml וחיבור ישיר

המסך מורכב משני מקטעים פשוטים בתוך `VerticalStackLayout`:

1. **מקטע דפדוף ותצוגת פריט יחיד:** כפתורי הקודם/הבא, כותרת, ערוץ, משך, כתובת אינטרנט (URL), סטטוס צפייה וכפתור שינוי סטטוס.

2. **מקטע הוספת סרטון:** שדות `Entry` עם `Two-Way Binding` וכפתור שמירה.

```
<!-- URL - פריט הנוכחי כולל כתובת ה -->
<Label Text="{Binding CurrentVideo.Title, StringFormat='{0} : כותרת:}'" FontAttributes=
<Label Text="{Binding CurrentVideo.ChannelName, StringFormat='{0} : ערוץ:}'" />
<Label Text="{Binding CurrentVideo.FormattedDuration, StringFormat='{0} : משך:}'" />
<Label Text="{Binding CurrentVideo.Url, StringFormat='{0} : כתובת אינטרנט:}'" TextColor="
<Label Text="{Binding CurrentVideo.WatchedStatusText, StringFormat='{0} : סטטוס:}'" />
```

```
<!-- DI Container ללא (Code-Behind) חיבור ישיר ושקוף בקוד האחורי -->
```

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
        BindingContext = new MainViewModel(new MockVideoService());
    }
}
```

3. שאלות חשיבה ואתגרים לתלמיד

שאלה 1: מנגנון `CanExecute`

מדוע השימוש ב- `Command` עם מתודת `CanExecute` עדיף בהרבה על בדיקה ידנית באירוע לחיצה (`Button_Clicked`)?

שאלה 2: פריט בודד לעומת אוסף (לקראת השלב הבא)

האפליקציה הנוכחית מציגה סרטון אחד בכל רגע נתון. מה הקושי המרכזי בחוויית המשתמש באופן זה, ואיזה רכיב של MAUI יפתור זאת כאשר נעבור לעבודה עם אוספים?

שאלה 3: הרחבת הפונקציונליות (משימת אתגר אישית)

הוסיפו כפתור "מחק סרטון" (`DeleteCommand`) אשר ימחק את הסרטון הנוכחי מהשירות, יעדכן את הרשימה ויעביר את התצוגה לסרטון שלפניו. אילו בדיקות תצטרכו להגדיר ב- `CanExecute` של פקודה זו?